House Edge

Participant guide · TAMU Datathon 2026

November 7–8, 2026 · All times Central

Use the slot-machine logs to predict the odds of the next payout: **0×, 1×, 2×, 5×, or 20×**. You'll submit a CSV of probabilities and the code needed to recreate it. Your score depends on how close those probabilities are to the machine's actual odds.

Start with the participant kit released by organizers. The repo is a project reference and may require access; you don't need repo access to compete. Practice data and the slot-machine demo are separate from the competition.

PLEASE DO NOT REGISTER THE SAME PERSON ON MULTIPLE TEAMS. YOUR REGISTRATION WILL BE REJECTED

Data:
https://drive.google.com/file/d/1WXrMzPXHv_ZuwpKvdRrcKYoJaMR5J6VV/view?usp=sharing

MANDATORY TEAM REGISTRATION FORM:
https://docs.google.com/forms/d/e/1FAIpQLSfFI3_new12AA2qpamG_fqRz6RnhdRHqnsoWAenEsNqB8VOJw/viewform?usp=sharing&ouid=100967425964110292916

MAIN ROUND SUBMISSION FORM:

https://docs.google.com/forms/d/e/1FAIpQLSfv2WV_Fz7NHstRPN2TGi3pvMAFrcQuvs2SkQL1DqpdLebhFg/viewform?usp=sharing&ouid=100967425964110292916

NOTE: Check your spam inbox for confirmation/rejection for registration.

Dates and links

Round two opens Saturday at 9 PM and closes Sunday at 11 AM. You'll have **14 hours** to work with the new data, including overnight and Sunday morning. Main submissions close when round two opens.

What	When or where
Event hacking begins	Saturday, November 7 at 11 AM
Main submissions close	Saturday, November 7 at 9 PM

Round-two data releases and submissions open	Saturday, November 7 at 9 PM
Round-two submissions close	Sunday, November 8 at 11 AM
Event hacking ends	Sunday, November 8 at noon
Judging begins	Sunday, November 8 at 12:30 PM
Participant kit	Download link to be posted with the challenge announcement
Upload page	Link to be announced
Help	Challenge help channel to be announced

Times above are **Central (America/Chicago)**. The upload page shows times in your browser's timezone. The event schedule lists the wider event activities; the two submission deadlines above are specific to House Edge.

Upload early! Both files **must** arrive and pass the initial file checks before the deadline. A receipt confirms they've been accepted; the code check can (and will) finish later.

What to predict

Each query asks for **one machine's next spin**, at its `cutoff` tick. Return five probabilities in this order: `0, 1, 2, 5, 20`. They should add up to 1.

For example, `[0.60, 0.20, 0.12, 0.06, 0.02]` gives a 60% chance of no payout and a 2% chance of a 20× payout. These probabilities also describe your uncertainty, so you don't need a separate confidence interval.

Every spin costs one chip. Payouts include the original wager: 0× loses the chip, 1× returns it, and 5× returns five chips, for a net gain of four. The competition scores your predictions, not the number of chips won.

Use whichever model you want. The starter runs on a CPU with Python; a neural network or GPU is optional. You don't need to build an agent or choose bets. The interactive demo is just a way to try the idea out and doesn't affect your score.

1. Getting started

Download and fully unzip the **main participant kit**, then open a terminal in the extracted folder. Keep the scripts, `slot_forecasting/`, and `public/` together. Run the commands from the folder containing `run_starter.py`.

You'll need **Python 3.11 or newer**. The starter and checker use the standard library, so there's nothing else to install. You don't need Docker for these steps.

```
python --version
```

If your system uses `python3` or `py`, use that instead of `python` below.

```
python run_starter.py --out outputs
python check_submission.py outputs/submission.csv
python check_submission.py outputs/validation_predictions.csv --split validation
python build_entry.py --starter outputs --out entry.zip
```

What these commands do

The starter trains a baseline and compares six methods on validation data. **It saves predictions from `pooled` by default**, even if another method scores better. You can choose another method with `--method`.

The two checker commands check your test file and score your validation file. A local check doesn't upload anything or reveal your final test score.

The last command packages your saved starter model and code. Upload **`outputs/submission.csv` and `entry.zip`** as two separate files. The `--starter` option works with `run_starter.py` outputs. For your own model or the experiment runner, follow the packaging instructions in Section 7.

File	What it contains
`submission.csv`	Test predictions to upload.
`validation_predictions.csv`	Predictions you can score locally.
`baseline.json`	The saved model.
`run.json`	Settings, comparisons, runtime, file hashes, and reproduction checks.
`validation_score.json`	Detailed validation scores.
`comparison/`	Validation predictions from each baseline.

Keep this first run. Use a new output folder for each experiment so you can go back to an earlier model. If you prefer notebooks, `starter.ipynb` covers the same steps; you'll need your own Jupyter installation.

2. The data

The kit includes a README, `RULES.md`, `DATA.md`, `EXPERIMENTS.md`, a submission README template, starter scripts, and the `public/` data folder.

Folder	What's inside	Use it for
`public/train/`	Queries, histories, observed next-spin payouts, and condition labels.	Training your model.
`public/validation/`	Queries, histories, observed payouts, true probabilities, conditions, and reference forecasts.	Comparing models and choosing settings.
`public/test/`	Queries and histories, without answers or condition labels.	Making your final predictions.

Each split includes a `sample_submission.csv`. Check `public/metadata.json` for the data version, season, round, points, and sizes. `SHA256.json` lists hashes of the original kit files.

Terms used in the files

Term	Meaning
Query	One request for five payout probabilities.
Episode	Related machines observed at the same cutoff.
Tick	A time step in the gameplay logs.
Label / outcome	One observed payout: 0, 1, 2, 5, or 20.
Target probabilities	The simulator's actual odds. Public for validation, hidden for test.

Condition	Stable, short history, drift, or shared change. Used to compare performance.
Adaptation	The name used in the files and upload page for round two.

Input columns

`queries.csv` contains:

Column	Meaning
`query_id`	Copy this ID into your prediction CSV.
`episode_id`	The group of related machines.
`machine_id`	The machine within that episode. Group histories by both IDs.
`machine_type`	`steady`, `balanced`, or `jackpot`.
`signal`	An observed feature: -1, 0, or 1.
`cutoff`	The tick to predict. Use inputs from earlier ticks only.

`history.csv` contains `episode_id, machine_id, tick, payout`. Training and validation also have `labels.csv`, which maps each `query_id` to its observed next payout, named `outcome`.

Output columns always follow payout order **0, 1, 2, 5, 20**. Write probabilities between 0 and 1, not percentages.

Using the history

- For cutoff 200, use ticks below 200. Another machine's result at tick 200 is unavailable too.
- If you make a training example for tick 150, use inputs from below 150. The payout at 150 can be its label, but not an input.
- A missing tick is unknown, not a zero payout. A two-tick lag isn't necessarily the second-most-recent row.
- Use other machines' earlier results only within the same episode.
- Condition labels are available for training and validation only. Your test predictor must work without them.

Train, validation, and test contain separate episodes. Keep whole episodes together if you make your own splits; related machines shouldn't appear on both sides of an evaluation split.

Cutoffs range from 192 to 256. Long histories include every earlier tick. Short histories include 40 recent consecutive ticks and 24 sampled older ticks, so always check the actual tick values.

Past payouts can interact at delays of up to eight ticks. Machines of the same type can respond differently, even in opposite directions. Win chances and payout sizes can change, sometimes across several machines together. These are patterns in the challenge's simulator, not a claim about real casino machines.

See `DATA.md` for details. Use validation answers to compare ideas, but remember that repeated tuning can overfit that data.

3. Your model

Try another baseline

```
python run_starter.py --method recent --out recent-run
```

```
python check_submission.py recent-run/validation_predictions.csv --split validation
```

```
python check_submission.py recent-run/submission.csv
```

The included methods are `global`, `context`, `empirical`, `recent`, `pooled`, and `pooled_recent`. You're free to use other methods and libraries.

Write your own script

Save this as `my_model.py` in the kit folder. It loads the data, fits a model, and writes validation and test predictions. Replace the model with your own when you're ready.

```
from pathlib import Path
from slot_forecasting.baseline import FrequencyForecaster
from slot_forecasting.io import load_inputs, read_labels, write_submission


DATA = Path("public")
OUT = Path("custom-run")
if OUT.exists():
    raise SystemExit("Choose a new output folder before running again.")


train_queries, train_history = load_inputs(DATA / "train")
labels = read_labels(DATA / "train/labels.csv", train_queries)
model = FrequencyForecaster(method="pooled_recent", half_life=24)
model.fit(train_queries, train_history, labels)
```

```
for split, filename in (
    ("validation", "validation_predictions.csv"),
    ("test", "submission.csv"),
):
    queries, history = load_inputs(DATA / split)
    predictions = model.predict(queries, history)
    write_submission(OUT / filename, predictions)
```

Then run:

```
python my_model.py
python check_submission.py custom-run/validation_predictions.csv --split validation
python check_submission.py custom-run/submission.csv
```

Here, `predict()` returns a dictionary from each `query_id` to five probabilities. The writer checks the numbers and keeps their precision; the checker makes sure every required query appears exactly once.

Save your dependency versions, seeds, settings, and trained model files. This example runs locally; to submit it, you'll also need a script that accepts the data and output paths in Section 7. Include any project modules it imports in your ZIP.

4. Scoring

Main: 80 points. Round two: 20 points. Add them for your final score. All points come from forecasts. There are no points for a writeup or presentation, but your code must reproduce your results and follow the rules for you to qualify for awards.

We compare your probabilities with the simulator's actual probabilities. That keeps a randomly drawn jackpot from deciding the score. Some machine state is hidden, so don't expect to recover the exact odds from the logs.

A simple example

If you predict `[0.60, 0.20, 0.12, 0.06, 0.02]` and the true probabilities are `[0.50, 0.25, 0.15, 0.08, 0.02]`, the sum of squared differences is `0.0138`. It stays the same whichever payout happens on that spin.

Assign a reasonable chance to each payout. Picking only the most likely outcome throws away information and can hurt your score when several outcomes are plausible.

How the points are calculated

1. Add the five squared probability errors for each query.

2. Average within each episode, then across episodes in each condition.
3. Divide each condition's error by the frozen pooled baseline's error for that condition, with a minimum denominator of `1e-9`.
4. Average the four ratios to get `R`.

```
R = average across conditions of error / max(reference_error, 0.000000001)
round points = round maximum / (1 + R)
final points = main points + adaptation points
```

The four conditions are **stable, short history, drift, and shared change**. Each counts equally. Each round has its own frozen reference model.

Example	Main	Round two	Total
Same error as the reference (`R = 1` in both rounds)	40	10	50
Half the normalized error (`R = 0.5` in both rounds)	53.33	13.33	66.67
Exact probabilities (`R = 0` in both rounds)	80	20	100

These are scoring examples, not predictions of how teams will do. Points aren't accuracy percentages: 60 points doesn't mean you guessed 60% of spins correctly.

Use the checker's validation points, or `validation.contest.technical_points` in `run.json`, to compare models. The older generic `score` field is a different diagnostic and isn't used for ranking. Higher points and lower errors are better.

5. Experiments

Save your baseline results, change one thing, and compare on the same validation episodes. Look at each condition as well as the total. A shorter history window might help changing machines and hurt stable ones.

Idea	First thing to try
Recent spins matter more	Compare history windows or give recent observations more weight.
Short histories are noisy	Blend a machine's estimate with an average from similar machines.

Order matters	Add earlier payouts or pairs of payouts at different tick lags.
Patterns change	Compare short and long memory, or try a change detector.
Machines share patterns	Use summaries of other machines' earlier results in the episode.
Models make different mistakes	Blend their probabilities and choose weights using validation data.

Run the included experiments

```
python run_experiments.py --out experiments-01
```

```
python check_submission.py experiments-01/submission.csv
```

This tries 12 settings. It chooses a model using half the validation episodes in each condition, then checks that model on the other half. It writes `submission.csv`, `validation_predictions.csv`, and `experiments.json`. Edit `starter_candidates()` in `slot_forecasting/experiments.py` to add your own candidates.

Once you start tuning against that second half, it stops being an independent check. Keep whole episodes together and look for future information accidentally getting into your features before trusting a large improvement.

Keep a short log for yourselves: what changed, the score, runtime, and whether you kept it. It isn't graded. Sequence models and other more involved approaches are welcome, but the score only depends on the forecasts. Leave time to package and test the model you choose. `EXPERIMENTS.md` has more ideas.

6. Round two

New data: Saturday, November 7 at 9 PM Central.

Submission deadline: Sunday, November 8 at 11 AM Central.

Main submissions close at 9 PM Saturday. Everyone then gets the second kit, with fresh training data, validation answers, and test histories. Some patterns have changed. You have **14 hours** to update your model and submit the new predictions.

You can reuse main-round data, code, and saved models. Refit the same model or try something different. Round two is scored directly out of 20; deliberately weakening your main entry won't earn an improvement bonus.

1. Keep your final main files and receipt.
2. Extract the new kit into a separate folder. Check that `public/metadata.json` says `adaptation`.
3. Copy over your code and model files without overwriting the new `public/` data.
4. Run your existing approach on the new validation data before changing it.
5. Check the new test CSV, package the matching code, and upload under **Adaptation**.

A suggested work plan

When	What to do
Saturday, soon after 9 PM	Download the kit and get a first valid run. Check that your code uses the new data.
Saturday night	Compare a few changes on the new validation set. Save a working entry before stopping.
Sunday morning	Review results, choose a model, and upload with time to fix problems.
Before 11 AM Sunday	Save your receipt and check the selected version and reproduction status.

The deadline is the same whether you work overnight or in the morning. Uploads must be accepted before 11 AM; code checks may finish afterward.

Your main CSV won't work for round two because the query IDs are different. A missing or invalid round-two entry earns zero for that round. You need a valid main entry for an overall ranking. Exact tied totals share a rank; organizers will announce how prize ties are handled before kickoff.

7. Submitting

The upload link will be announced. You'll use the House Edge submission page to upload two files for each round: your test CSV and the code ZIP that recreates it.

Upload your files

1. Open the submission page and enter your team's token. Keep the token private.
2. Choose **Main** or **Adaptation**. Check the deadline and timezone shown on the page.
3. Select your test CSV and code ZIP, then click **Send entry**. Wait for the receipt.

4. Download the receipt JSON. It records the round, time received, file hashes, selected version, and code-check status.
5. Click **Refresh** to check the result. `passed` means the recreated probabilities match within the allowed tolerance.

If the connection drops, check your receipts before starting another upload. Retrying the same files from the unchanged page uses the same upload key and won't create a duplicate. The limits are 20 accepted entries per team per round and 12 upload attempts per minute.

Your latest accepted entry counts, even if its code check later fails. A malformed CSV or ZIP is rejected and doesn't replace your previous entry. Once a replacement is accepted, there's no automatic fallback. Fix and upload it again before the deadline, or contact organizers under the announced correction policy.

Status	Meaning
`queued` / `running`	Files accepted; the code check is waiting or running.
`passed`	Recreated probabilities match. Eligibility checks still apply.
`mismatch`	The code ran, but its predictions differ from your CSV.
`failed`	The command failed, timed out, exceeded a limit, or produced invalid output. Open Run details.
`infrastructure_error`	An organizer needs to check the service. Your original files are stored and can be retried.

The receipt time decides whether an entry arrived before the deadline. Hidden test points are released with final results, not during uploads.

Package a custom model

For the starter, use the `build_entry.py --starter` command in Section 1. For your own model, prepare a folder like this:

```
my-entry/
  README.md
  reproduce.json
  predict.py
  model files and configuration
  required source modules and offline dependencies
```

The last two lines describe files to include; use their actual filenames. Keep the forecast CSV separate. Leave out unrelated data, secrets, and temporary outputs.

Put this in `reproduce.json` at the folder's root:

```
{
  "image": "python",
  "command": ["python", "predict.py", "--data", "/data", "--out",
"/output/submission.csv"]
}
```

Your `predict.py` must accept `--data` and `--out`. The command is an argument list, not a shell string. It runs from `/submission`, where your code is read-only. The paths are:

- `/data`: the current round's public data, read-only.
- `/main_data`: main-round public data, available during adaptation.
- `/tmp`: writable scratch space, limited to 128 MiB.
- `/output`: writable output space, limited to 32 MiB. Write the final CSV to `/output/submission.csv`.

The runner has no network access, hidden test answers, or organizer credentials. It doesn't receive your uploaded forecast CSV as an input.

The standard `python` environment uses Python 3.13 with the standard library. Other approved environments will be listed on the upload page. For extra libraries, use an offered environment or include compatible Linux wheels and an offline setup script. Don't rely on downloads during the run. Ask organizers early about other environments or GPUs.

Test your command locally from the source folder, replacing the example paths with your kit's public data folder and a fresh output path:

```
python predict.py --data "/absolute/path/to/kit/public" --out
"/absolute/path/to/check/submission.csv"
```

Run the checker on that output from the kit folder. Make sure it matches the CSV you plan to upload. Saving a trained checkpoint is usually easier than retraining during the check, especially if training is slow or uses randomness.

Package the source folder from the kit folder:

```
python build_entry.py --source my-entry --out custom-entry.zip
```

Keep the ZIP outside `my-entry/`. It must open directly to `README.md` and `reproduce.json`, without an extra enclosing folder. Limits are 5,000 entries, 128 MiB compressed, and 512 MiB unpacked. Symlinks, encrypted files, unsafe paths, and duplicate paths are rejected.

Default runtime limits are **30 minutes, two CPU cores, and 16 GiB RAM**. The largest allowed difference per probability is `1e-8` after CSV normalization. The final CSV may be at most 16 MiB. Check the upload page for the frozen event settings.

Item	Include
Team and round	Team ID and `main` or `adaptation`.
Forecast CSV	Test predictions for this round.
Code ZIP	README, reproduce.json, source, dependencies, settings, and saved model files.
README	Dependency versions, seeds, data paths, training details, and the command to recreate the CSV.

Use `SUBMISSION_README.md` as a starting point. A short setup README is enough. There is no separate report or video for this scoring process; follow any additional event-wide requirements announced by organizers.

CSV format

```
query_id,p_0,p_1,p_2,p_5,p_20
example_query,0.60,0.20,0.12,0.06,0.02
```

Replace the example ID with the actual IDs from this round's `public/test/queries.csv`.

- Include every test query exactly once, with no extras. Row order doesn't matter.
- Use exactly these six columns, without a dataframe index or extra columns.
- Probabilities must be finite numbers between 0 and 1 and add up to 1 within `1e-6`.
- Save as UTF-8 CSV. Renaming an Excel workbook won't convert it.
- Keep enough decimal places to preserve your probabilities.

Before you upload

- Check the exact CSV you're uploading against the right round's test data.
- Recreate it from a fresh folder using the code ZIP and README.
- Include all dependencies, model files, and settings. Don't rely on an old notebook session or paths that only exist on your computer.
- If you change the model or preprocessing, rebuild both the CSV and ZIP.
- Leave time for the upload and initial file checks before cutoff.

After you upload

Save the receipt and confirm that the intended version is selected. Check the reproduction status. Main entries can't be replaced after 9 PM Saturday; adaptation entries can't be replaced after 11 AM Sunday.

Organizers score the accepted forecasts on private test answers and combine both rounds. Code reproduction and data-use checks must pass before awards. Pending checks and service failures must be resolved before the final ranking. Use the announced correction process if something goes wrong; don't assume a late file sent by chat or email will count.

8. Troubleshooting

Setup and modeling

Problem or question	What to try
`python` isn't found	Try `python3` or `py`, and check that it's version 3.11+.
Script or module not found	Fully unzip the kit and run from its folder. Keep `slot_forecasting/` beside the scripts.
Missing data or `Incomplete public dataset`	Check that `public/` fully extracted. Use `--data "path/to/public"` if it's elsewhere.
`query_id mismatch`	Check the round and split. Validation files need `--split validation`; uploads need test IDs.
Probabilities don't add up to one	Check column order, normalize each row, and avoid rounding too much. Find the cause of any NaNs.
Extra columns	Remove dataframe indices. With pandas, use `index=False`.
Output folder already exists	Pick a new `--out` folder.
Points differ from the `score` field	Use `technical_points` for the round score. The older diagnostic has a different scale.

Can I use coding assistants or automated tuning?	Yes. You're responsible for the results and for following the data rules.
Do I need RL, a neural network, or a GPU?	No. Any model that produces the required probabilities can compete.
Can I use outside data?	Use organizer-released data. Private test answers and organizer files are off limits.
Can I calculate my final score locally?	You can score validation predictions. Test answers stay private.
Does the slot-machine demo count?	No. It uses practice odds; submit your CSV to compete.

Uploads and code checks

Problem	What to check
No receipt	Refresh your receipt list. Without an accepted receipt, the upload isn't confirmed. Save any error and contact organizers.
Wrong query IDs	Main, adaptation, validation, and test have different IDs. Use the correct kit and split.
ZIP rejected	Check the size limits and root files. Use `build_entry.py` to package it.
Module not found on the runner	Include your modules and compatible offline dependencies. Your local installs aren't copied over.
Can't write a file	Use `/tmp` for scratch and `/output` for results. `/submission` and `/data` are read-only.
`mismatch`	Check the checkpoint, preprocessing, column order, precision, and seeds against the uploaded CSV.

Timeout	Load a saved checkpoint, remove unnecessary work, and stay within the runtime budget.
`queued` or `running`	Your files were accepted. Refresh later; replacing an entry just to restart the queue won't help.
`infrastructure_error`	Give organizers the receipt ID so they can retry the stored files.
A replacement fails	That replacement still counts. Fix it and upload again before cutoff; older entries aren't automatically restored.
Deadline passed	Follow the announced correction or outage process.
Second kit locked	It opens at 9 PM Saturday, regardless of when your main entry finishes its code check.

When asking for help, include the command, Python version, round, and exact error. Keep your token private. Share code or model files only if you intend others to see them.